



GRAINE FOURNIE AQUIPLANTS

Modélisation de la Base de Données

Adresse : Route de la Crau, 13630 Eyragues.

Téléphone : 04 90 94 59 31.

SIREN : 393 606 702.

Sommaire

VI. Modélisation de la Base de Données	3
7.1 Introduction à la méthode MERISE	3
7.2 Dictionnaire des données	3
7.3 Modèle Conceptuel de Données (MCD)	9
7.4 Modèle Logique de Données (MLD)	11
7.5 Modèle Physique de Données (MPD)	12
7.6 Justifications du choix du modèle.....	15

VI. Modélisation de la Base de Données

7.1 Introduction à la méthode MERISE

Pour la conception de la base de données du projet Graine Fournie AQUIPLANTS, j'ai suivi la démarche MERISE. Cette méthode se déroule en trois étapes progressives :

- MCD (Modèle Conceptuel de Données) : représentation des entités métier et de leurs associations, indépendamment de toute considération technique.
- MLD (Modèle Logique de Données) : traduction du MCD en tables relationnelles avec identification des clés primaires et étrangères.
- MPD (Modèle Physique de Données) : adaptation au SGBD choisi (MySQL 8.0) avec les types SQL exacts, les index et les contraintes d'intégrité.

Cette progression permet de s'assurer que la base de données correspond bien aux besoins du projet, tout en étant adaptée à MySQL 8.0.

7.2 Dictionnaire des données

Le dictionnaire des données liste tous les champs de chaque table de l'application.

Table UTILISATEUR

Attribut	Type	Contrainte	Description
Id_utilisateur	Entier	Clé primaire Auto-incrémentation	Identifiant unique de l'utilisateur
nom	Texte	Obligatoire	Nom de l'utilisateur
MDP_crypte	Texte	Obligatoire	Mot de passe haché (bcrypt)
role	Texte	Obligatoire	Rôle : Admin ou employé
Prenom	Texte	Obligatoire	Prénom du compte
Email	Texte	Obligatoire	Email du compte

Table CLIENT

Attribut	Type	Contrainte	Description
Id_client	Entier	Clé primaire Auto-incrémentation	Identifiant unique du client
nom_client	Texte	Obligatoire	Nom du client
prenom_client	Texte	Obligatoire	Prénom du client

Table PLANT

Attribut	Type	Contrainte	Description
Id_plant	Entier	Clé primaire Auto-incrémentation	Identifiant unique du plant
nom_plant	Texte	Obligatoire	Nom commercial du plant
Nom_espece	Texte	Obligatoire	Nom scientifique ou commun de l'espèce

Table UV

Attribut	Type	Contrainte	Description
id_uv	Entier	Clé primaire Auto-incrémentation	Identifiant unique de l'UV
nom_uv	Texte	NOT NULL	Nom ou référence de l'UV
nb_graine_par_motte	Entier	Obligatoire	Nombre de graines semées par motte

Table GF_Client (graine fournie – réception client)

Attribut	Type	Contrainte	Description
id_gf_client	Entier	Clé primaire Auto-incrémentation	Identifiant unique de la réception
reference_gf	Texte	Obligatoire, Unique	Référence du lot
quantite_disponible	Entier	NOT NULL	Stock total cumulé
Seuil_alerte	Entier	Obligatoire	Quantité minimale avant le déclenchement d'une alerte
nom_client	Texte	Obligatoire	Nom du client (affichage rapide)
id_client	Entier	Clé Étrangère de CLIENT	Client fournisseur
id_plant	Entier	Clé Étrangère PLANT	Variété concernée

Table GF_Histo_Client (historique des semis réalisées)

Attribut	Type	Contrainte	Description
id_histo	Entier	Clé primaire Auto-incrémentation	Identifiant unique de l'historique
quantite_semee	Entier	Obligatoire	Quantité de graines effectivement semées
date_semis	Date	Obligatoire	Date à laquelle le semis a été réalisé
nom_uv	Texte	Obligatoire	Nom de l'UV concernée
nb_graine_par_motte	Entier	Obligatoire	Nombre de graines par motte lors du semis
id_gf_client	Entier	Clé Étrangère de GF_CLIENT	Référence à la réception concernée
id_uv	Entier	Clé Étrangère de UV	Référence à l'UV concernée

Table Commande_A_Semer

Attribut	Type	Contrainte	Description
id_commande	Entier	Clé primaire Auto-incrémentation	Identifiant unique de la commande
quantite_a_semer	Entier	Obligatoire	Quantité de graines à semer pour cette commande
date_semis	Date	Obligatoire	Date prévue pour réaliser le semis
date_livraison	Date	Obligatoire	Date prévue de livraison au client
id_uv	Entier	Clé Étrangère de UV	UV associée à cette commande
id_client	Entier	Clé Étrangère de CLIENT	Client demandeur de la commande

Table Emplacement

Attribut	Type	Contrainte	Description
id_emplacement	Entier	Clé primaire Auto-incrémentation	Identifiant unique de l'emplacement
lettre_etagere	Texte	Obligatoire	Lettre de l'étagère (A, B, C ou D)
numero_etage	Entier	Obligatoire	Numéro de l'étage (1, 2, 3 ou 4)
id_gf_client	Entier	Clé Étrangère de GF_CLIENT	Sachet rangé à cet emplacement

Table LOG

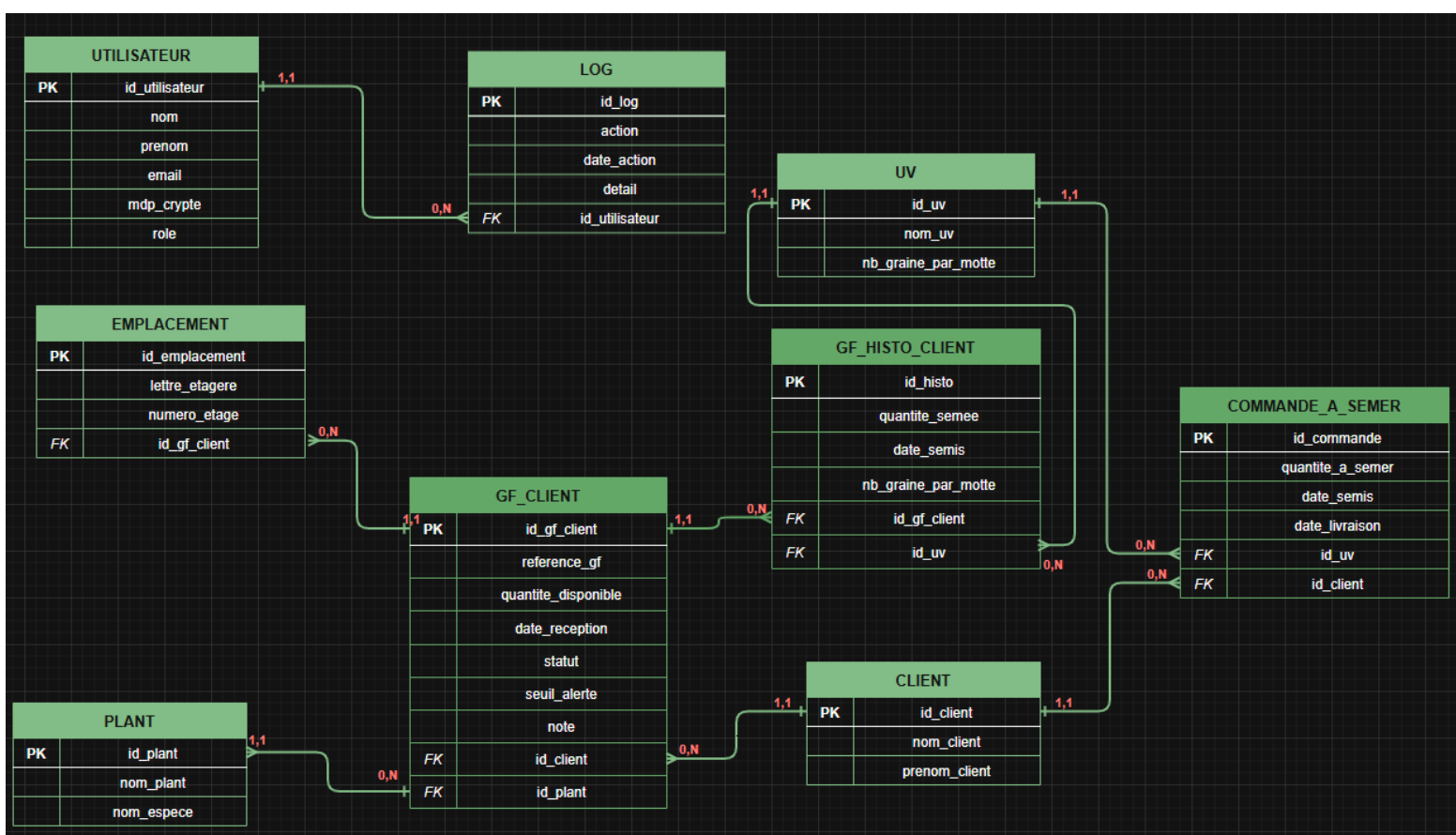
Attribut	Type	Contrainte	Description
id_log	Entier	Clé primaire Auto-incrémentation	Identifiant unique de log
action	Texte	Obligatoire	Type d'action (connexion, ajout, suppression...)
date_action	Date + Heure	Obligatoire	Date et heure de l'action
Detail	Texte	Optionnel	Description détaillée de l'action
id_utilisateur	Entier	Clé Étrangère d'utilisateur	Utilisateur qui a effectué l'action

Table HISTO_GF_DEPOSEE

Attribut	Type	Contrainte	Description
id_histo_depot	Entier	Clé primaire Auto-incrémentation	Identifiant unique du dépôt
quantite_deposee	Entier	Obligatoire	Quantité déposée lors de ce dépôt
date_reception	Date + Heure	Obligatoire	Date du dépôt
statut	Texte	Optionnel	à_traiter ou rangé
note	Texte	Optionnel	Observation sur le dépôt
id_gf_client	Entier	Clé Étrangère GF_CLIENT	Lot concerné

7.3 Modèle Conceptuel de Données (MCD)

Le Modèle Conceptuel de Données (MCD) représente l'ensemble des entités métier de l'application et leurs relations, indépendamment de toute contrainte technique. Il a été réalisé selon la méthode MERISE et constitue la première étape de la modélisation, avant la traduction en tables relationnelles.



PK : Primary Key – Clé primaire

FK : Foreign Key - Clé étrangère.

Chaque entité correspond à un objet du domaine métier d'AQUIPLANTS : les clients qui apportent leurs graines, les sachets reçus, les emplacements de

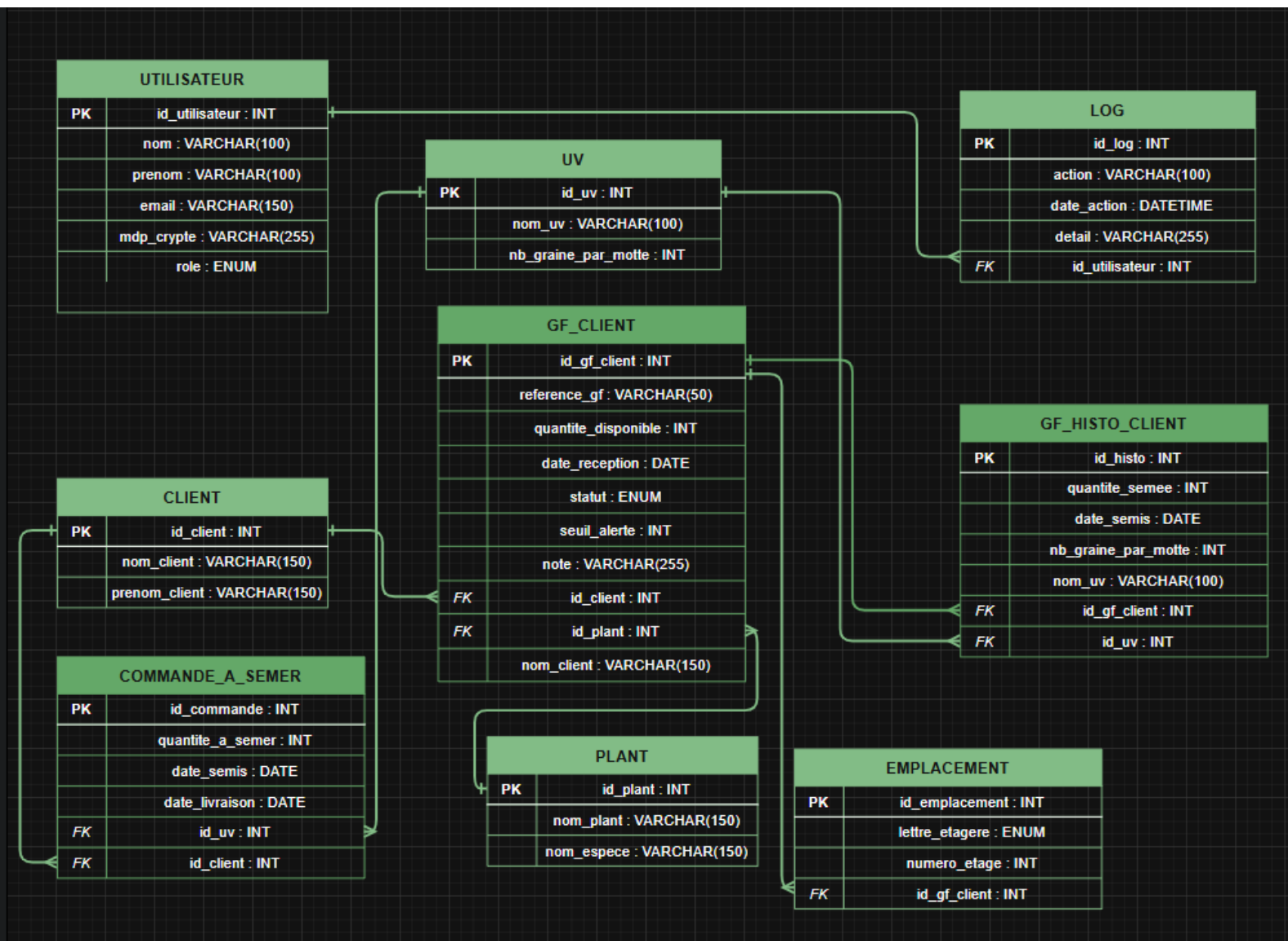
stockage, les commandes à semer, ou encore les utilisateurs de l'application. Les associations entre entités sont accompagnées de leurs cardinalités, qui définissent le nombre de fois qu'une entité peut être liée à une autre.

Entités identifiées :

- UTILISATEUR — représente les employés et administrateurs de l'application
- CLIENT — représente les clients qui fournissent les graines
- PLANT — représente les espèces de plants cultivées
- UV — représente les unités de valeur (lignes de semis avec un nombre de graines par motte)
- GF_CLIENT — représente une réception de graines fournies par un client
- GF_HISTO_CLIENT — représente l'historique des semis réalisés à partir des graines fournies
- COMMANDE_A_SEMER — représente une commande de semis à réaliser

Associations et cardinalités :

- CLIENT (1,1) ----< fournit >---- (0,N) GF_CLIENT — Un client peut fournir plusieurs lots de graines. Un lot appartient à un seul client.
- GF_CLIENT (1,1) ----< génère >---- (0,N) GF_HISTO_CLIENT — Un lot de graines fourni peut donner lieu à plusieurs semis historisés.
- UV (1,1) ----< est utilisée dans >---- (0,N) GF_HISTO_CLIENT — Une UV peut apparaître dans plusieurs historiques de semis.
- UV (1,1) ----< est associée à >---- (0,N) COMMANDE_A_SEMER — Une UV est ciblée par une commande de semis.
- CLIENT (1,1) ----< passe >---- (0,N) COMMANDE_A_SEMER — Un client peut passer plusieurs commandes.



7.4 Modèle Logique de Données (MLD)

Le Modèle Logique de Données (MLD) est la traduction du MCD en tables relationnelles. Les entités deviennent des tables, les associations disparaissent et sont remplacées par des clés étrangères (FK) qui matérialisent les liens entre les tables.

Le MLD du projet AQUIPLANTS est composé de 9 tables : UTILISATEUR, CLIENT, PLANT, UV, GF_CLIENT, GF_HISTO_CLIENT, COMMANDE_A_SEMER, EMPLACEMENT et LOG. Chaque table possède une clé primaire (PK) auto-incrémentée et les relations entre tables sont assurées par des clés étrangères (FK).

7.5 Modèle Physique de Données (MPD)

Base de données : MySQL 8.0 | Encodage : utf8mb4

```
-- =====
-- MPD - GRAINE FOURNIE AQUIPLANTS
-- Base de données : MySQL 8.0
-- =====

CREATE DATABASE IF NOT EXISTS graine_fournie_aquiplants
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;

USE graine_fournie_aquiplants;

-- -----
-- Table : UTILISATEUR
-- -----
CREATE TABLE utilisateur (
    id_utilisateur INT NOT NULL AUTO_INCREMENT,
    nom VARCHAR(100) NOT NULL,
    prenom VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL,
    mdp_crypte VARCHAR(255) NOT NULL,
    role ENUM('admin', 'employe') NOT NULL DEFAULT 'employe',
    CONSTRAINT pk_utilisateur PRIMARY KEY (id_utilisateur),
    CONSTRAINT uq_utilisateur_email UNIQUE (email)
);

-- -----
-- Table : CLIENT
-- -----
CREATE TABLE client (
    id_client INT NOT NULL AUTO_INCREMENT,
    nom_client VARCHAR(150) NOT NULL,
    prenom_client VARCHAR(150) NOT NULL,
    CONSTRAINT pk_client PRIMARY KEY (id_client)
);

-- -----
-- Table : PLANT
-- -----
CREATE TABLE plant (
    id_plant INT NOT NULL AUTO_INCREMENT,
    nom_plant VARCHAR(150) NOT NULL,
    nom_espece VARCHAR(150) NOT NULL,
    CONSTRAINT pk_plant PRIMARY KEY (id_plant)
);
```

PROJET GRAINE FOURNIE AQUIPLANTS - JALON 3

```
-- Table : UV
-----
CREATE TABLE uv (
    id_uv          INT          NOT NULL AUTO_INCREMENT,
    nom_uv         VARCHAR(100) NOT NULL,
    nb_graine_par_motte INT      NOT NULL,
    CONSTRAINT pk_uv PRIMARY KEY (id_uv)
);

-- Table : GF_CLIENT
-----
CREATE TABLE gf_client (
    id_gf_client    INT          NOT NULL AUTO_INCREMENT,
    reference_gf     VARCHAR(50)  NOT NULL,
    quantite_disponible INT      NOT NULL DEFAULT 0,
    seuil_alerte     INT          NOT NULL DEFAULT 0,
    nom_client       VARCHAR(150) NOT NULL,
    id_client        INT          NOT NULL,
    id_plant         INT          NULL,
    CONSTRAINT pk_gf_client PRIMARY KEY (id_gf_client),
    CONSTRAINT uq_gf_reference UNIQUE (reference_gf),
    CONSTRAINT fk_gf_client_client FOREIGN KEY (id_client)
        REFERENCES client (id_client)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_gf_client_plant FOREIGN KEY (id_plant)
        REFERENCES plant (id_plant)
        ON DELETE SET NULL ON UPDATE CASCADE
);

-- Table : HISTO_GF_DEPOSEE
-----
CREATE TABLE histo_gf_deposee (
    id_histo_depot    INT          NOT NULL AUTO_INCREMENT,
    quantite_deposee  INT          NOT NULL,
    date_reception    DATE         NOT NULL,
    statut            ENUM('a_traiter', 'range') NOT NULL DEFAULT 'a_traiter',
    note              VARCHAR(255) NULL,
    id_gf_client       INT          NOT NULL,
    CONSTRAINT pk_histo_gf_deposee PRIMARY KEY (id_histo_depot),
    CONSTRAINT fk_histo_depot_gf_client FOREIGN KEY (id_gf_client)
        REFERENCES gf_client (id_gf_client)
        ON DELETE CASCADE ON UPDATE CASCADE
);

-- Table : GF_HISTO_CLIENT
-----
CREATE TABLE gf_histo_client (
    id_histo         INT          NOT NULL AUTO_INCREMENT,
    quantite_semee    INT          NOT NULL,
    date_semis       DATE         NOT NULL,
    nom_uv           VARCHAR(100) NOT NULL,
    nb_graine_par_motte INT      NOT NULL,
    id_gf_client      INT          NOT NULL,
    id_uv            INT          NOT NULL,
    CONSTRAINT pk_gf_histo_client PRIMARY KEY (id_histo),
    CONSTRAINT fk_histo_gf_client FOREIGN KEY (id_gf_client)
        REFERENCES gf_client (id_gf_client)
        ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_histo_uv FOREIGN KEY (id_uv)
        REFERENCES uv (id_uv)
        ON DELETE RESTRICT ON UPDATE CASCADE
);
```

PROJET GRAINE FOURNIE AQUIPLANTS - JALON 3

```
-- Table : COMMANDE_A_SEMER
-----
CREATE TABLE commande_a_semer (
  id_commande      INT              NOT NULL AUTO_INCREMENT,
  quantite_a_semer INT              NOT NULL,
  date_semis       DATE             NOT NULL,
  date_livraison   DATE             NOT NULL,
  id_uv            INT              NOT NULL,
  id_client        INT              NOT NULL,
  CONSTRAINT pk_commande_a_semer PRIMARY KEY (id_commande),
  CONSTRAINT fk_commande_uv FOREIGN KEY (id_uv)
    REFERENCES uv (id_uv)
    ON DELETE RESTRICT ON UPDATE CASCADE,
  CONSTRAINT fk_commande_client FOREIGN KEY (id_client)
    REFERENCES client (id_client)
    ON DELETE RESTRICT ON UPDATE CASCADE
);

-----
-- Table : EMBLACEMENT
-----
CREATE TABLE emplacement (
  id_emplacement  INT              NOT NULL AUTO_INCREMENT,
  lettre_etagere  ENUM('A', 'B', 'C', 'D') NOT NULL,
  numero_etage    INT              NOT NULL,
  id_gf_client    INT              NOT NULL,
  CONSTRAINT pk_emplacement PRIMARY KEY (id_emplacement),
  CONSTRAINT uq_emplacement UNIQUE (lettre_etagere, numero_etage, id_gf_client),
  CONSTRAINT fk_emplacement_gf_client FOREIGN KEY (id_gf_client)
    REFERENCES gf_client (id_gf_client)
    ON DELETE CASCADE ON UPDATE CASCADE
);

-----
-- Table : LOG
-----
CREATE TABLE log (
  id_log          INT              NOT NULL AUTO_INCREMENT,
  action          VARCHAR(100)     NOT NULL,
  date_action     DATETIME         NOT NULL DEFAULT CURRENT_TIMESTAMP,
  detail          VARCHAR(255)     NULL,
  id_utilisateur  INT              NOT NULL,
  CONSTRAINT pk_log PRIMARY KEY (id_log),
  CONSTRAINT fk_log_utilisateur FOREIGN KEY (id_utilisateur)
    REFERENCES utilisateur (id_utilisateur)
    ON DELETE RESTRICT ON UPDATE CASCADE
);
```

7.6 Justifications du choix du modèle.

La modélisation de la base de données élaboré en prenant en compte des exigences de la pépinière AQUIPLANTS.

La table GF_CLIENT constitue la table centrale du projet. Elle stocke une ligne par client et par variété, avec la quantité disponible totale cumulée. La table HISTO_GF_DEPOSEE enregistre chaque dépôt physique avec sa date, son statut et sa quantité, permettant de garder la traçabilité complète tout en ayant un stock consolidé.

La table GF_HISTO_CLIENT garde une trace de chaque semis fait à partir d'un sachet. Cela permet de savoir combien de graines ont été semées, quand, et avec quelle UV. C'est la réponse directe au problème de traçabilité identifié dans le cahier des charges.

La table COMMANDE_A_SEMER relie un client à une UV pour planifier les semis à venir. Elle couvre le suivi des commandes clients qui était jusqu'ici géré manuellement sur papier.

La table LOG enregistre chaque action faite par un utilisateur dans l'application, ce qui permet de savoir qui a fait quoi et quand.

Le schéma évite les doublons : chaque information est stockée une seule fois et les liens entre tables sont assurés par les clés étrangères.